Distributional Semantics

CS 115B: Fundamentals of NLP — Neural Networks III

James Pustejovsky

Brandeis University

Spring 2026

Today's Lecture

1. The Problem of Meaning
2. Count-Based Methods
3. Pointwise Mutual Information
4. Dimensionality Reduction: SVD
5. Word2Vec
6. GloVe
7. Properties of Word Embeddings
8. Bias in Word Embeddings
9. Evaluation and Usage
10. Summary

How Do We Represent Word Meaning?

For NLP systems, we need to represent words mathematically.

Option 1: Symbolic / One-hot encoding

$$\text{cat} = [0, 0, 1, 0, 0, \ldots, 0] \quad \text{dog} = [0, 0, 0, 1, 0, \ldots, 0]$$

Problems:

- Vectors are **orthogonal**: $\mathbf{v}_{\text{cat}} \cdot \mathbf{v}_{\text{dog}} = 0$
- No notion of **similarity** — "cat" is as different from "dog" as from "democracy"
- Dimensionality = vocabulary size (50K–100K+)
- No generalization: learning about "cat" tells us nothing about "dog"

What we want: Representations where **similar words have similar vectors**.

The Distributional Hypothesis

J.R. Firth (1957)

"You shall know a word by the company it keeps."

The idea: Words that occur in similar **contexts** have similar **meanings**.

Examples:

- "I had _____ for breakfast" ⇒ oatmeal, eggs, toast, cereal, ...
- "The _____ barked loudly" ⇒ dog, puppy, hound, ...
- "She drove her _____ to work" ⇒ car, truck, van, ...

Intuition: If two words appear in the same contexts, they probably mean similar things.

This is the foundation of **distributional semantics**.

Historical Context

The idea has deep roots:

Zellig Harris (1954): Distributional Structure

"Difference of meaning correlates with difference of distribution."

Ludwig Wittgenstein (1953): Philosophical Investigations

"The meaning of a word is its use in the language."

Modern incarnations:

- 1990s: Latent Semantic Analysis (LSA)
- 2000s: Topic models, PMI-based methods
- 2013: Word2Vec (Mikolov et al.)
- 2014: GloVe (Pennington et al.)
- 2018+: Contextual embeddings (ELMo, BERT, GPT)

Intuition: Learning from Context

Consider these sentences with a nonsense word:

- "A bottle of **tezgüino** is on the table."
- "Everybody likes **tezgüino**."
- "**Tezgüino** makes you drunk."
- "We make **tezgüino** out of corn."

Intuition: Learning from Context

Consider these sentences with a nonsense word:

- "A bottle of **tezgüino** is on the table."
- "Everybody likes **tezgüino**."
- "**Tezgüino** makes you drunk."
- "We make **tezgüino** out of corn."

What can we infer?

- Comes in bottles
- People consume it
- Has intoxicating effects
- Made from corn

Intuition: Learning from Context

Consider these sentences with a nonsense word:

- "A bottle of **tezgüino** is on the table."
- "Everybody likes **tezgüino**."
- "**Tezgüino** makes you drunk."
- "We make **tezgüino** out of corn."

What can we infer?

- Comes in bottles
- People consume it
- Has intoxicating effects
- Made from corn

Tezgüino is a corn beer from Mexico!
We inferred this purely from **distributional context**.

Term-Document Matrix

Idea: Represent words by the documents they appear in.

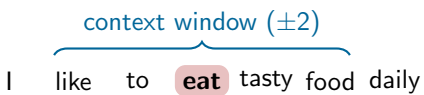
	Doc 1 (Sports)	**Doc 2** (Finance)	**Doc 3** (Sports)	**Doc 4** (Politics)
goal	5	0	3	1
score	4	2	6	0
market	0	8	0	2
stock	0	7	0	1
vote	0	0	0	9

- Each row is a word vector
- $\mathbf{v}_{\text{goal}} = [5, 0, 3, 1]$
- Similar words $\rightarrow$ similar row vectors
- Basis of *Latent Semantic Analysis* (LSA)

Term-Context Matrix (Word-Word)

Better idea: Represent words by the **words** they co-occur with.

Context: Words appearing within a window (e.g., ± 2 words)



For target word "eat":

Context words: {like, to, tasty, food}

Increment co-occurrence counts: $M[\text{eat}, \text{like}]{+}{+}$, etc.

Co-occurrence Matrix: Example

Tiny corpus:

"I like deep learning. I like NLP. I enjoy deep NLP."

Co-occurrence matrix (window size = 1):

	I	like	deep	learning	NLP	enjoy
I	0	2	0	0	0	1
like	2	0	1	0	1	0
deep	0	1	0	1	1	1
learning	0	0	1	0	0	0
NLP	0	1	1	0	0	0
enjoy	1	0	1	0	0	0

Word vectors: Each row is a representation of that word.

Problems with Raw Counts

Issue 1: Frequent words dominate
"the", "a", "of" co-occur with everything — not informative!

Issue 2: Counts are unbounded
Hard to compare: is 100 co-occurrences "a lot"?

Issue 3: Sparsity
Most word pairs never co-occur (count $= 0$)

Issue 4: High dimensionality
Vocabulary of 100K $\Rightarrow$ 100K-dimensional vectors

Solutions:

1. Reweight counts: **Pointwise Mutual Information (PMI)**
2. Reduce dimensions: **Singular Value Decomposition (SVD)**

Pointwise Mutual Information (PMI)

Key question: Do words w and c co-occur *more than expected by chance*?

PMI Definition

$$\text{PMI}(w, c) = \log_2 \frac{P(w, c)}{P(w) \cdot P(c)}$$

Interpretation:

- PMI > 0: Words co-occur *more* than expected (associated)
- PMI $= 0$: Words co-occur as expected (independent)
- PMI < 0: Words co-occur *less* than expected (dissociated)

Intuition: High PMI = the co-occurrence is **informative**.

PMI: Example Calculation

Corpus statistics: Total co-occurrences $N = 10^9$

	Count	**Probability**
"data" appears	100,000	$P(\text{data}) = 10^{-4}$
"science" appears	80,000	$P(\text{science}) = 8 \times 10^{-5}$
"data" + "science" together	5,000	$P(\text{data, science}) = 5 \times 10^{-6}$

Expected co-occurrence if independent:

$$P(\text{data}) \cdot P(\text{science}) = 10^{-4} \times 8 \times 10^{-5} = 8 \times 10^{-9}$$

PMI:

$$\text{PMI} = \log_2 \frac{5 \times 10^{-6}}{8 \times 10^{-9}} = \log_2 625 \approx 9.3$$

High PMI! Strong association between "data" and "science".

Positive PMI (PPMI)

Problem with PMI: Negative values are unreliable.

- Negative PMI requires observing *fewer* co-occurrences than expected
- But rare events are noisy
- Also: $\log(0) = -\infty$ when words never co-occur

Solution: Positive PMI (PPMI)

PPMI Definition

$$\text{PPMI}(w, c) = \max(0, \text{PMI}(w, c))$$

Simply clip negative values to zero.

PPMI matrices are the standard for count-based distributional semantics.
Also common: context distribution smoothing (raising context counts to power 0.75).

Dimensionality Reduction with SVD

PPMI matrices are still huge and sparse.
Solution: Singular Value Decomposition (SVD)

SVD

$$\mathbf{M} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^\top$$

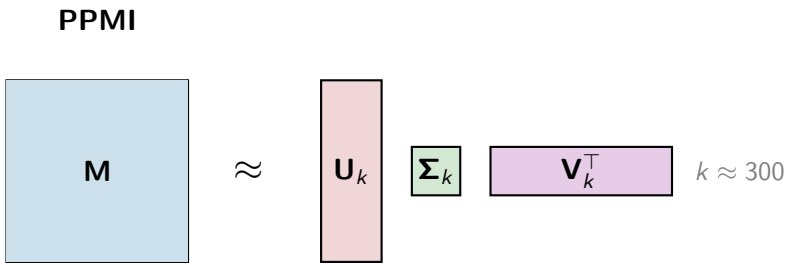
- $\mathbf{M}$: $|V| \times |V|$ PPMI matrix
- $\mathbf{U}$: $|V| \times |V|$ orthogonal matrix (left singular vectors)
- $\mathbf{\Sigma}$: diagonal matrix of singular values (ordered by magnitude)
- $\mathbf{V}$: $|V| \times |V|$ orthogonal matrix (right singular vectors)

Truncated SVD: Keep only top k dimensions

$$\mathbf{M} \approx \mathbf{U}_k\mathbf{\Sigma}_k\mathbf{V}_k^\top$$

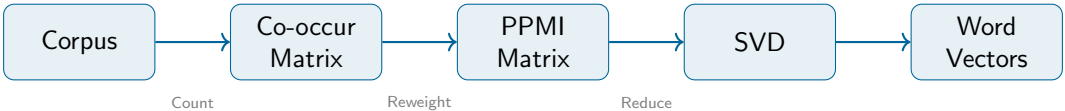
Word vectors: Rows of $\mathbf{U}_k$ (or $\mathbf{U}_k\mathbf{\Sigma}_k^{1/2}$), typically $k = 300$

SVD Visualization



Result: Dense k-dimensional vector for each word!

Count-Based Pipeline Summary



Pros:

- Mathematically principled (optimal low-rank approximation)
- Captures global statistics
- Deterministic

Cons:

- Expensive for large vocabularies (SVD is $O(|V|^3)$)
- Hard to add new words
- Doesn't scale well to web-scale data

Word2Vec: A Neural Approach

Mikolov et al. (2013) — Revolutionized NLP!

Key insight: Instead of counting, **predict** context from words (or vice versa).

Two architectures:

Skip-gram

Predict *context* from center word

"The cat **sat** on the mat"

$\Rightarrow$ predict: the, cat, on, the, mat

CBOW

(Continuous Bag of Words)

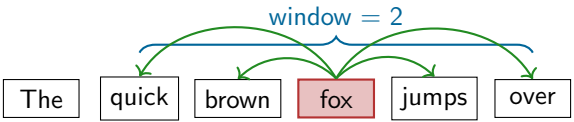
Predict *center word* from context

"The cat ____ on the mat"

$\Rightarrow$ predict: sat

Word vectors: Learned as parameters during training!

Skip-gram: The Idea



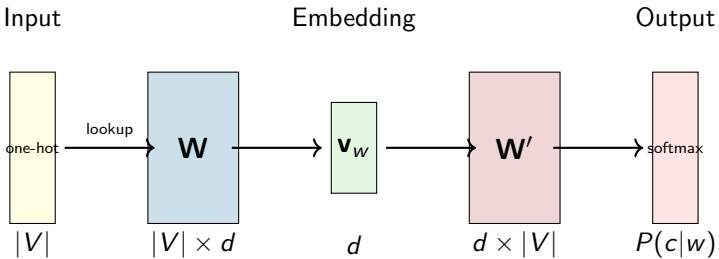
Task: Given "fox", predict "quick", "brown", "jumps", "over"

Training objective:

$$\max \sum_{t=1}^{T} \sum_{\substack{-c \le j \le c \\ j \neq 0}} \log P(w_{t+j} \mid w_t)$$

Maximize probability of context words given center word.

Skip-gram: Architecture



Two embedding matrices:

- **W**: Target word embeddings (rows are word vectors)
- **W′**: Context word embeddings

Final embeddings: use **W**, or average **W** and **W′**

The Softmax Problem

Probability of context word c given target w:

$$P(c \mid w) = \frac{\exp(\mathbf{v}'_c \cdot \mathbf{v}_w)}{\sum_{c' \in V} \exp(\mathbf{v}'_{c'} \cdot \mathbf{v}_w)}$$

Problem: The denominator sums over **entire vocabulary**!

- $|V|$ can be 100,000+ words
- Need to compute this for every training example
- Extremely expensive!

Solutions:

1. Hierarchical softmax (tree-structured vocabulary)
2. **Negative sampling** (much more popular)

Negative Sampling

Idea: Replace softmax with binary classification.

Task: Distinguish *real* (target, context) pairs from *fake* ones.

Positive examples:
Real co-occurrences from corpus

- (fox, quick) ✓
- (fox, brown) ✓
- (fox, jumps) ✓

Negative examples:
Random word pairs

- (fox, banana) ×
- (fox, democracy) ×
- (fox, purple) ×

For each positive pair, sample k negative pairs (typically $k = 5$–15)
Negatives sampled from smoothed unigram distribution: $P_n(w) \propto f(w)^{0.75}$

Negative Sampling Objective

Skip-gram with Negative Sampling (SGNS)

$$\mathcal{L} = \log \sigma(\mathbf{v}'_c \cdot \mathbf{v}_w) + \sum_{i=1}^{k} \mathbb{E}_{n_i \sim P_n} \left[\log \sigma(-\mathbf{v}'_{n_i} \cdot \mathbf{v}_w) \right]$$

Two terms:

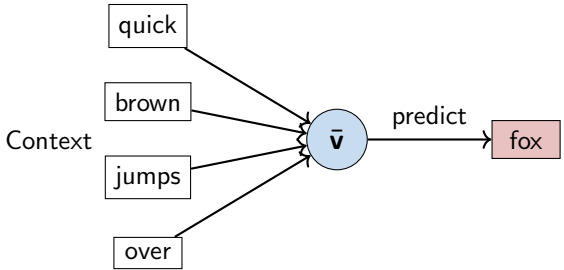
1. $\log \sigma(\mathbf{v}'_c \cdot \mathbf{v}_w)$: Push real context vectors **closer** to target
2. $\log \sigma(-\mathbf{v}'_n \cdot \mathbf{v}_w)$: Push random vectors **farther** from target

Efficiency: Only $k + 1$ computations per example instead of $|V|$!

Remarkable finding (Levy & Goldberg, 2014):
SGNS implicitly factorizes a shifted PMI matrix!

$$\mathbf{v}_w \cdot \mathbf{v}'_c \approx \text{PMI}(w, c) - \log k$$

CBOW: Continuous Bag of Words



CBOW: Average context embeddings $\rightarrow$ predict center word

Comparison:

	Skip-gram	**CBOW**
Training speed	Slower	Faster
Rare words	Better	Worse
Typical use	Default choice	Large datasets

Word2Vec Hyperparameters

Key choices:

Parameter	Typical Values
Embedding dimension d	100–300
Context window size c	5–10
Negative samples k	5–15
Subsampling threshold	10^{-3} to 10^{-5}
Minimum word count	5–10
Training epochs	5–15

Window size matters:

- Smaller window (± 2): More syntactic similarity
- Larger window (± 10): More topical/semantic similarity

GloVe: Global Vectors

Pennington et al. (2014): Combines count-based and prediction-based

Key insight: Co-occurrence *ratios* encode meaning!

	$P(\cdot \mid \text{ice})$	$P(\cdot \mid \text{steam})$	Ratio
solid	1.9×10^{-4}	2.2×10^{-5}	8.9
gas	6.6×10^{-5}	7.8×10^{-4}	0.085
water	3.0×10^{-3}	2.2×10^{-3}	≈ 1
fashion	1.8×10^{-5}	1.8×10^{-5}	≈ 1

"solid" is related to "ice" (high ratio)
"gas" is related to "steam" (low ratio)
"water" and "fashion" don't distinguish them (ratio ≈ 1)

GloVe Objective

Goal: Learn vectors whose dot product equals log co-occurrence:

$$\mathbf{v}_i^\top \mathbf{v}_j + b_i + b_j = \log X_{ij}$$

Weighted least squares:

$$J = \sum_{i,j=1}^{|V|} f(X_{ij}) \left(\mathbf{v}_i^\top \mathbf{v}_j + b_i + b_j - \log X_{ij} \right)^2$$

Weighting function:

$$f(x) = \begin{cases} (x/x_{\max})^{0.75} & \text{if } x < x_{\max} \\ 1 & \text{otherwise} \end{cases}$$

Downweights very frequent pairs; ignores pairs with $X_{ij} = 0$

Training: Stochastic gradient descent on non-zero entries

Word2Vec vs. GloVe: Summary

	Word2Vec (SGNS)	**GloVe**
Approach	Prediction-based	Count + optimization
Training data	Local windows (online)	Global co-occurrence matrix
Objective	Log-likelihood	Weighted least squares
Implicit matrix	Shifted PMI	Log co-occurrence

In practice: Both work well! Similar performance.

Pre-trained embeddings available:

- **Word2Vec:** Google News (100B words), 300d
- **GloVe:** Common Crawl (840B tokens), up to 300d
- **fastText:** Wikipedia, includes subword information

Measuring Similarity

Cosine similarity:

$$\cos(\mathbf{v}_a, \mathbf{v}_b) = \frac{\mathbf{v}_a \cdot \mathbf{v}_b}{\|\mathbf{v}_a\| \|\mathbf{v}_b\|}$$

Most similar words to "king" (GloVe):

Word	Cosine Similarity
kings	0.71
queen	0.65
prince	0.62
monarch	0.60
throne	0.59

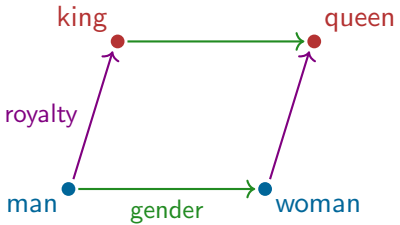
Finding similar words:

$$\text{most_similar}(w) = \arg\max_{w' \neq w} \cos(\mathbf{v}_w, \mathbf{v}_{w'})$$

The Analogy Property

Famous finding: Vector arithmetic captures semantic relationships!

$$\mathbf{v}_{\text{king}} - \mathbf{v}_{\text{man}} + \mathbf{v}_{\text{woman}} \approx \mathbf{v}_{\text{queen}}$$



Analogy task:

$$\arg\max_{w} \cos(\mathbf{v}_w, \mathbf{v}_b - \mathbf{v}_a + \mathbf{v}_c) \quad \text{for } a : b :: c :?$$

More Analogies

Syntactic:

- walking : walked :: swimming : **swam**
- good : better :: bad : **worse**
- big : biggest :: small : **smallest**

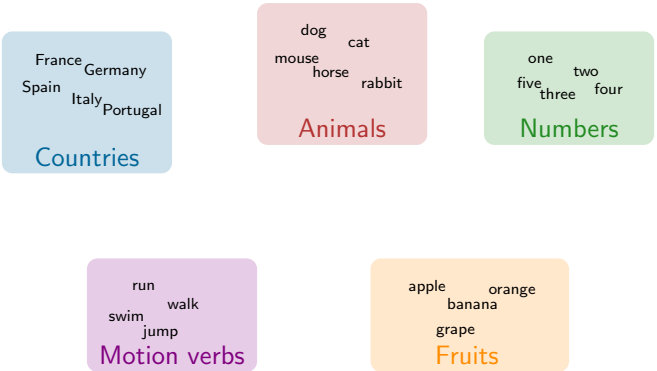
Semantic:

- Paris : France :: Tokyo : **Japan**
- Einstein : physicist :: Picasso : **painter**
- brother : sister :: nephew : **niece**

Caveats:

- Accuracy typically 40–70% (not perfect!)
- Often fails for rare words
- Can reflect stereotypes and biases

Visualizing Embeddings



t-SNE projection: semantically related words cluster together

Bias in Word Embeddings

Embeddings learn from text — including its biases.

Bolukbasi et al. (2016): "Man is to Computer Programmer as Woman is to Homemaker"

Problematic analogies:

- man : computer programmer :: woman : **homemaker**
- man : doctor :: woman : **nurse**
- he : engineer :: she : **teacher**

Why this matters:

- Embeddings used in downstream systems (search, hiring, recommendations)
- Can perpetuate or **amplify** societal biases
- Important area of ongoing research

Mitigation: Debiasing techniques exist but remain imperfect.

Evaluating Word Embeddings

Intrinsic evaluation: Test embeddings directly

1. **Word similarity:**
   - Datasets: WordSim-353, SimLex-999, MEN
   - Metric: Spearman correlation with human judgments

2. **Analogy:**
   - Dataset: Google analogy (19K questions)
   - Metric: Accuracy

Extrinsic evaluation: Use as features in downstream tasks

- Named Entity Recognition, Sentiment Analysis, Parsing, etc.
- **This is what really matters!**

Using Pre-trained Embeddings

Best practice: Don't train from scratch — use pre-trained!

Available embeddings:

- Word2Vec: Google News (100B words)
- GloVe: Common Crawl (840B tokens)
- fastText: Wikipedia (many languages, subword-aware)

Usage in neural models:

1. Initialize embedding layer with pre-trained vectors
2. **Option A:** Freeze (don't update during training)
3. **Option B:** Fine-tune (continue updating)

Fine-tuning usually helps if you have enough task data.

This is transfer learning! (Foreshadows BERT, GPT, etc.)

Limitations of Static Embeddings

One vector per word, regardless of context.

Polysemy problem: "bank"

- "I deposited money in the **bank**." (financial)
- "I sat on the river **bank**." (geographical)

Same vector for both!

Other issues:

- Out-of-vocabulary words have no embedding
- Rare words have poor embeddings
- Morphology not captured ("run" vs "running")

Solutions:

- Subword embeddings (fastText)
- **Contextual embeddings** (ELMo, BERT, GPT) — coming later!

Summary

Distributional Hypothesis:

- "You shall know a word by the company it keeps"
- Words in similar contexts have similar meanings

Count-based methods:

- Co-occurrence matrices $\rightarrow$ PMI/PPMI $\rightarrow$ SVD
- Principled, but doesn't scale well

Prediction-based methods:

- Word2Vec: Skip-gram / CBOW with negative sampling
- GloVe: Weighted matrix factorization of co-occurrences
- Fast, scalable, high-quality embeddings

Properties: Similarity, analogies, but also biases!

What's Next

This lecture: Word embeddings — dense representations of meaning

Next lectures: Multi-Layer Perceptrons (2 lectures)

- Using embeddings as neural network inputs
- Hidden layers for non-linear transformations
- Backpropagation algorithm
- Deep learning fundamentals

Coming later:

- Recurrent Neural Networks (sequences)
- Transformers and Attention
- Contextual embeddings (BERT, GPT)

Word embeddings are the foundation of neural NLP!